

Technical Knowledge & Interview Prep

1. Abstract Class vs Interface

- **Abstract Class:** A partial blueprint that can provide shared implementation (code, fields, properties). A class can inherit from **only ONE** base class (single inheritance).
- **Interface:** A contract that defines *what* properties/methods a class must have, without caring about its inheritance tree. A class can implement **MULTIPLE** interfaces.

Why use both (e.g. `IAuditableEntity` and `AuditableEntity`)?

- * **Interface alone:** You have to copy-paste the audit properties into every entity.
- * **Abstract Class alone:** You can't use it for classes that already inherit from something else (like `ApplicationUser` inheriting `IdentityUser`).
- * **Both together:** `IAuditableEntity` allows EF Core to track all entities uniformly. `AuditableEntity` gives normal models reusable code so you don't repeat properties across dozens of entities.

2. Record, Class, and Struct

- **Class (Reference Type):** Stored on the Heap. Two separate class instances with identical data are **not equal** by default. Best for stateful objects, entities, and services.
- **Struct (Value Type):** Stored on the Stack. Copied by value. Best for small, lightweight, short-lived data (`DateTime` , `Guid`).
- **Record (Reference Type by default):** Stored on the Heap. A `class` supercharged with built-in **value-equality**, immutability, and concise syntax. Best for DTOs and API payloads.

3. LINQ (Language Integrated Query)

- **Deferred vs Immediate Execution:**
 - Deferred (`Where` , `Select` , `OrderBy`): Query does not run until enumerated (e.g., in a `foreach`).
 - Immediate (`ToList` , `Count` , `First`): Executes the query immediately and materializes results.
- **`IEnumerable` vs `IQueryable` :**
 - `IEnumerable` : Runs in application memory (C# RAM).
 - `IQueryable` : Translates to SQL (via Expression Trees) and runs on the Database Server.

- **Any() vs Count() > 0** : `Any()` is faster (stops at the first element, translates to `IF EXISTS`). `Count()` counts all elements first.
- **FirstOrDefault() vs SingleOrDefault()** : Use `First` when you just want one. Use `Single` when having more than 1 is a data integrity error (e.g., searching by unique Email).

4. Async & Concurrency

- **What async/await does**: It frees up the thread while waiting for I/O operations. It increases server scalability, not individual query speed.
- **Danger of .Result or .Wait()** : Blocking a thread while waiting (sync-over-async) causes **Thread-Pool Starvation** and wraps errors in `AggregateException`. Always use "async all the way up".
- **Task.WhenAll** : Runs independent I/O tasks concurrently to reduce overall wait time.
- **Task vs ValueTask** : `Task` allocates on the heap. `ValueTask` is a struct (stack) used for high-throughput hot paths where results are frequently synchronous (e.g., cache hits).
- **CancellationToken** : Pass it down to EF Core to abort queries instantly if the user disconnects, saving CPU and DB resources.

5. Dependency Injection (Lifetimes)

- **Transient**: New instance every time it is requested.
- **Scoped**: One instance per HTTP Request. (Ideal for `DbContext`, Repositories).
- **Singleton**: One instance for the entire app lifetime. (Caches, config).
- **Captive Dependency**: When a Singleton injects a Scoped service (like `DbContext`). The `DbContext` lives forever, causing thread-safety crashes and memory leaks. Fix: Use `IServiceScopeFactory`.

6. EF Core Deep Dive

- **Tracking States**: `Added` (INSERT), `Unchanged` (Nothing), `Modified` (UPDATE), `Deleted` (DELETE), `Detached` (Untracked).
- **AsNoTracking()** : Prevents EF from keeping memory snapshots. Improves performance for read-only queries and prevents accidental writes.
- **Include vs Projection (Select)** : `Include` loads full entity graphs (over-fetching). `Select` generates SQL asking only for specific columns needed for DTOs.

- **Optimistic Concurrency (RowVersion):** A `[Timestamp] byte[]` column that updates on every change. Prevents "lost updates" (User A overwriting User B) by throwing `DbUpdateConcurrencyException`.
- **Filtered Unique Index:** `HasFilter("[DeletedAt] IS NULL")` ensures uniqueness only for active records, allowing soft-deleted users to free up their email.
- **Explicit Transactions:** Use `BeginTransactionAsync` when multiple `SaveChangesAsync()` calls or external services must succeed or fail together.

7. EF Core Migrations

- **What it is:** Version control for your database schema. It bridges the gap between C# Code-First entities and SQL tables.
- **How it works:** `Up()` applies changes forward. `Down()` rolls them back.
- **Tracking:** EF uses the `__EFMigrationsHistory` table to know which migrations have run.

8. EF Core vs Dapper

- **EF Core:** Best for domain writes, transactions, and tracking.
- **Dapper:** A Micro-ORM where you write raw SQL. Blazing fast. Best for read-heavy, reporting, or complex multi-table join queries where hand-crafted SQL is clearer.

9. CQRS, MediatR, and Pipeline Behaviors

- **CQRS (Command Query Responsibility Segregation):** Separates Write operations (Commands) from Read operations (Queries). Does not require two databases.
- **MediatR:** A library implementing the Mediator pattern. Decouples Controllers from Handlers (Controller sends a message to MediatR, MediatR routes it to the Handler).
- **Pipeline Behaviors:** Middleware for MediatR. Runs before and after Handlers. Used for cross-cutting concerns (e.g., `ValidationBehavior` running `FluentValidation` on every request).
- **Custom Exceptions:** Domain-specific exceptions (`NotFoundException` , `BadRequestException`) mapped to HTTP Status Codes (404, 400) via Global Error Handling.

10. Repository Pattern & DataContext

- **Repository Pattern:** An abstraction layer between business logic and data access. Allows for easy Unit Testing (mocking) and centralizes complex query logic.
- **DataContext (DbContext):** EF Core's representation of a database session (Unit of Work). It holds `DbSet` s, manages DB connections, tracks changes, and translates LINQ to SQL.

11. JWT & Security

- **JWT Structure:** `Header.Payload.Signature` .
- **The Golden Rule:** The payload is Base64 encoded, **not encrypted**. Never put secrets in a JWT. The signature only proves authenticity.
- **Role vs Policy Authorization:** Role is simple ("Is Admin"). Policy is flexible ("Must be > 18 years old").
- **Access vs Refresh Tokens:** Access Tokens are short-lived for API requests. Refresh Tokens are long-lived, securely stored, and used to silently get new Access Tokens without re-login.

12. Object-Oriented Programming (OOP) Pillars

- **Encapsulation:** Hiding internal state. (e.g., `ValidationBehavior._validators` is private).
- **Abstraction:** Hiding complexity behind a contract. (e.g., `ICurrentUserService` hides HTTP context extraction).
- **Inheritance:** Code reuse. (e.g., `AuditableEntity : BaseEntity`).
- **Polymorphism:** Treating different objects uniformly. (e.g., `ChangeTracker.Entries<IAuditableEntity>()` iterates over `Post` , `User` , etc., dynamically).

13. Nullable Reference Types (NRT)

- Introduced in C# 8 to fix the "Billion Dollar Mistake" (`NullReferenceException`).
- `string` = Will never be null (Compiler warns if you assign null).
- `string?` = Might be null (Compiler forces you to check before using).